AE646 Interim Report: Fourier Neural Operator for Parametric Darcy Flow

Team: Aryaman Srivastava **Prepared:** August 2026 (ahead of the Stage 2 deadline) **Course:** AE646 - Scientific Machine Learning for Fluid Mechanics

1. Dataset & Preprocessing (Complete)

Real PDEBench 2D Darcy Flow (β=1.0), downloaded from the official source (DaRUS/Uni Stuttgart, DOI 10.18419/darus-2986) and verified by checksum against PDEBench's published data manifest — 10,000 genuine samples at native 128×128 resolution.

- Equation: `-div(kappa * grad(u)) = f`, `f = beta = 1.0`, `u = 0` on the boundary
- `kappa`: piecewise-constant permeability in {0.1, 1.0} (thresholded Gaussian random field — PDEBench's actual Darcy convention)

Preprocessing pipeline (`src/preprocess.py`):

1. Reproducible (seed=42) non-overlapping subset: 1000 samples for train/val, 200 for test
2. Downsample 128×128 → 64×64 (stride-2 subsampling) for the main pipeline
3. Split 900 train / 100 val
4. Normalize using train statistics only (global mean/std)
5. Add coordinate channels: input = [permeability, x_coord, y_coord]
6. Native 128×128 fields for the 200 test samples saved separately, for a zero-shot super-resolution check planned for the final stage

Verification: shapes checked, normalization stats saved (`norm_stats.json`), no data leakage (train/val/test are disjoint, stats from train only), reproducible via a fixed seed.

2. Baseline Implementation (Complete)

MLP Baseline:

```
MLPBaseline(input_channels=3, output_channels=1, height=64, width=64,
hidden_dims=[2048, 2048, 2048], activation=GELU)
```

- Parameters: **41,953,280** (~42.0M)
- Flattens spatial dimensions → dense layers → unflattens; no spatial inductive bias

Training config: AdamW, lr=1e-3, weight_decay=1e-4, cosine annealing, 100 epochs, batch size 16, MSE loss.

Results (test set, 200 real PDEBench samples, relative L2 in physical units):

- Mean rel L2: **0.0820**, median 0.0695, std 0.0456 (best epoch 99)

- Unlike an earlier internal run on a much smaller (~200-sample) self-generated dataset, the MLP does **not** collapse into severe overfitting here — the larger, real 900-sample training set gives it enough signal to learn a reasonable (if spatially unstructured) fit. It is still clearly outperformed by the FNO below.

---

3. Initial SciML Implementation (Complete)

FNO Architecture:

```
FNO2d(input_channels=3, output_channels=1, width=64, modes=12, n_layers=4)
```

- Lift: Linear(3→64); 4 × [SpectralConv2d(64,64,modes=12) + 1×1 Conv + LayerNorm + GELU]; Project: Linear(64→128→1)
- Parameters: **4,744,449** (~4.7M)

Spectral convolution: FFT → multiply learned complex weights (modes × modes, two weight tensors for the low-frequency corners) → IFFT. Verified: output shapes correct, gradients flow, unit-tested against a fixed input/output shape contract (`tests/test_models.py`).

Training config: same as MLP (AdamW, cosine, 100 epochs).

Results (test set, physical units):

- Mean rel L2: **0.0521**, median 0.0398, std 0.0451 (best epoch 92)
- Clearly outperforms the MLP baseline (1.6× lower mean error) with **8.8× fewer parameters**

---

4. Preliminary Quantitative Results

Model	Mean Rel L2	Median	Std	Parameters
MLP	0.0820	0.0695	0.0456	42.0M
FNO	**0.0521**	0.0398	0.0451	4.7M

Key observations:

- FNO achieves lower error than MLP with far fewer parameters, consistent with the literature's claim that spectral convolutions provide a useful spatial inductive bias
- Both models show a similar error spread (std ≈ 0.045); a handful of test samples with near-uniform permeability fields are harder for both models (see error tails)
- Metric note: relative L2 is computed after denormalizing predictions/targets back to physical pressure units (see `physical_rel_l2` in `src/train.py`) — computing it directly on standardized (zero-mean/unit-std) fields would give a different, not directly literature-comparable number, since subtracting a constant changes the norm of the target but not the norm of (prediction - target)

---

5. Issues Faced & Next Steps

Issues:

1. The originally-planned data download script pointed at an incorrect URL; the correct official PDEBench source (DaRUS, not the URL initially assumed) was located and verified by checksum before proceeding
2. Local machine RAM/storage became a constraint once training three configurations with the full 900-sample real dataset; training was moved to a remote GPU machine for the larger runs, with results cross-checked against a local run for consistency
3. FNO test error (~0.05) is closer to, but still above, the ~0.01–0.02 typically reported in FNO papers — plausible contributors (PDEBench's different permeability convention, smaller training set, model capacity) are being investigated for the final report

Next steps:

1. Train the larger FNO configuration (width=128, modes=20, 6 layers) for the accuracy/parameter-count trade-off study
2. Zero-shot super-resolution: evaluate the trained FNO at native 128×128 against real PDEBench ground truth (not a synthetic proxy)
3. Measure real inference speed (FNO vs MLP vs a finite-difference Darcy solver)
4. Comparison plots and final report

---

6. Code Reproducibility

Structure:

```
src/
|-- download_data.py # real PDEBench download
|-- preprocess.py # subset selection, downsampling, normalization, split
|-- models.py # MLP, SpectralConv2d, FNO2d
|-- train.py # training loop (physical-unit relative-L2 metric)
|-- evaluate.py # metrics, visualizations
|-- superres_eval.py # zero-shot super-resolution (planned for final stage)
|-- benchmark_speed.py # inference-speed benchmark (planned for final stage)
`-- compare_comprehensive.py
```

Configs: `configs/fno.yaml`, `configs/mlp.yaml`, `configs/fno_improved.yaml` **Seed:** 42 (data subset, split, model init) **Checkpoints:** best-validation model saved with full state (model, optimizer, scheduler, epoch, metrics)

Run commands:

```
python src/download_data.py
python src/preprocess.py
python src/train.py --config configs/fno.yaml
python src/train.py --config configs/mlp.yaml
python src/evaluate.py --config configs/fno.yaml --checkpoint results/run_001/best_model.pt
```

---

7. AI Tool Use Declaration

AI tools (Claude) were used for: implementing and debugging the data pipeline and models, locating and verifying the correct official PDEBench data source, and drafting this report. All code was run, inspected, and the results independently reproduced by the student across two different machines.